

Module 08: Planning — Algorithms, Pathfinding, and Project Design

Learning Objectives

1. Write a **planning algorithm** (like BFS pathfinding) that finds a path from start to goal.
2. Design a **project plan** by breaking a big coding project into smaller steps.
3. Connect algorithmic planning to Active Inference: evaluating future possibilities before acting.

Introduction

Planning is the superpower of thinking ahead. In code, planning means writing algorithms that consider multiple possibilities *before* choosing the best one. This is the difference between a dumb bot (one that tries random directions) and a smart one (one that computes the shortest path first, then follows it). This final module brings together everything you have learned to design and build a capstone coding project.

Key Concepts

1. Breadth-First Search: Finding the Shortest Path

The **BFS (Breadth-First Search)** algorithm finds the shortest path in a grid:

```

from collections import deque

def bfs(grid, start, goal):
    queue = deque([(start, [start])])
    visited = set()

    while queue:
        current, path = queue.popleft()
        if current == goal:
            return path

        for neighbor in get_neighbors(current, grid):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None # No path found

```

BFS explores all possibilities in order of distance — it is guaranteed to find the shortest path. In Active Inference terms, BFS is an agent that mentally simulates every possible future path and selects the one with the lowest expected "cost" (fewest steps).

2. Pseudocode: Planning Before Coding

Before writing real code, write **pseudocode** — a plain-English version of your algorithm:

1. Start at the entrance
2. Look at all neighboring cells
3. For each neighbor that is not a wall and not visited:
 - a. Remember the path to get there
 - b. Add it to the "to explore" list
4. Repeat until you reach the exit
5. Return the path

Pseudocode is *planning* in its purest form: you think through all the steps before you act. This reduces bugs (prediction errors!) because you catch logic problems early.

3. Project Design: The Big Picture

For your capstone project, plan before you code:

1. **Define the goal:** What will your project do?
2. **Break it into parts:** What classes or functions do you need?
3. **Order the parts:** Which ones depend on which? Build foundations first.
4. **Estimate time:** How long will each part take?
5. **Identify risks:** What might go wrong? What are you uncertain about?

This is Expected Free Energy in action: you are evaluating possible futures (what could go well, what could go wrong) and choosing the plan that minimizes risk while maximizing learning.

Activities



Activity 1: Maze Solver

Write a BFS pathfinding algorithm that solves a maze represented as a 2D grid. Display the maze, the path, and the number of cells explored. Challenge: compare BFS to a random walker — how many extra steps does the random walker take?

Activity 2: Capstone Project Plan

Plan your end-of-unit project. It should combine at least 4 of the 8 Active Inference spine concepts. Write:

1. A 1-paragraph description of what it does
2. A list of all functions you will need
3. A dependency diagram (which functions call which)
4. A timeline (what you will build each day)

Present your plan to the class and get feedback before coding!

Summary

Planning in code means evaluating possibilities before acting. BFS finds the shortest path by systematically exploring all options. Pseudocode catches bugs before they happen. Project planning applies the same logic to larger creative work. These skills — thinking ahead, considering alternatives, and choosing wisely — are the computational embodiment of Active Inference.

References

- Bhargava, A. (2016). *Grokking Algorithms*. Chapter 6.